MVSR ENGINEERING COLLEGE

Cloud Infrastructure and DevOps Engineer Domain

PROJECT REPORT

URL Shortener using Lambda + DynamoDB

A Serverless URL Shortening System using Amazon API Gateway,
AWS Lambda, Amazon DynamoDB and AWS CDK

Submitted by:

Akshith Reddy Ponna

Roll Number: 2451-23-751-028

GitHub Repository:
https://github.com/Akshith7707/URL-shortner

Tools: AWS CDK, Lambda, API Gateway, DynamoDB
Verification Mode: Local zero-cost testing with DynamoDB Local

2. Acknowledgment

I would like to express my sincere gratitude to my project guide and faculty members for their continuous support, guidance and encouragement throughout the development of this project.

I also thank the Department of Computer Science and Engineering for providing the academic environment and resources required to complete this work. Finally, I thank my friends and family for their support during the project.

3. Abstract

Cloud computing enables developers to build scalable applications without managing physical servers. This project implements a serverless URL shortener on Amazon Web Services using Amazon API Gateway, AWS Lambda and Amazon DynamoDB. The system accepts a long URL, stores a short code mapping, and redirects users from the short link to the original destination.

Infrastructure is defined using AWS Cloud Development Kit (CDK) with TypeScript, following least-privilege IAM design and CloudWatch logging/monitoring. To avoid additional AWS spending during development and evaluation, the application was verified end-to-end using DynamoDB Local and a local API stand-in that invokes the same Lambda handlers.

Test results confirm successful health checks, short-URL creation, HTTP 301 redirection and HTTP 404 handling for unknown codes. The project demonstrates practical DevOps and cloud engineering skills: architecture design, Infrastructure as Code, secure IAM, automated testing and documentation.

4. Table of Contents

1. Cover Page
2. Acknowledgment
3. Abstract
4. Table of Contents
5. Introduction
6. Literature Survey
7. Problem Statement
8. Objectives
9. Existing System
10. Proposed System
11. System Architecture
12. Technologies Used
13. Implementation Methodology
14. System Testing
15. Results and Discussion
16. Advantages and Limitations
17. Applications
18. Future Scope
19. Conclusion
20. References

5. Introduction

5.1 Background

Modern applications increasingly rely on cloud platforms to reduce operational overhead and improve scalability. Serverless computing is a cloud model where the cloud provider runs the compute layer and developers focus on business logic. AWS Lambda, API Gateway and DynamoDB are commonly used together for lightweight HTTP APIs.

5.2 Need for the Project

Long URLs are difficult to share in messages, documents and social media. A URL shortener:

- Converts a long URL into a compact short code
- Stores the mapping in a durable database
- Redirects clients from the short URL to the original URL
- Can track usage with counters and logs

5.3 Scope

This project covers design, Infrastructure as Code, application logic, security (IAM least privilege), monitoring design, local verification and documentation. Production custom domains and authentication are listed as future enhancements.

6. Literature Survey

6.1 Serverless Computing

Serverless platforms execute code in response to events and scale automatically. Compared with always-on virtual machines, serverless can reduce idle cost for spiky workloads such as URL shortening APIs.

6.2 API Gateway and Lambda Integration

Amazon API Gateway exposes HTTPS endpoints and integrates with Lambda using proxy integration. HTTP APIs provide a lower-cost option for simple REST-style routes, which suits create and redirect operations in this project.

6.3 NoSQL Key-Value Storage

Amazon DynamoDB is a managed NoSQL database optimized for key-based access. A short-code lookup is a natural partition-key access pattern and provides single-digit millisecond reads at scale.

6.4 Infrastructure as Code

AWS CDK allows infrastructure to be defined in general-purpose languages and synthesized to CloudFormation. This improves repeatability, reviewability and alignment between application and infrastructure code.

7. Problem Statement

Manual or single-server URL shortener designs suffer from operational complexity, weak scaling characteristics and inconsistent deployment practices. There is a need for an automated, serverless design that can create short links, redirect users reliably, enforce least-privilege access and be deployed reproducibly using Infrastructure as Code.

8. Objectives

- Design a serverless URL shortener architecture on AWS
- Implement create and redirect APIs using Lambda and DynamoDB
- Define infrastructure using AWS CDK (TypeScript)
- Apply least-privilege IAM permissions
- Configure CloudWatch logging and error alarms in the stack
- Validate the system with automated and smoke tests
- Document architecture, results and future cloud deployment steps

9. Existing System

Traditional URL shorteners are often hosted on a single application server with a relational database. Limitations include:

- Server management overhead (patching, capacity planning)
- Single point of failure if only one instance is used
- Difficulty reproducing infrastructure across environments
- Broader IAM permissions than necessary in many student demos

10. Proposed System

The proposed system is a serverless URL shortener. Clients call an HTTP API. Lambda functions perform create and redirect logic. DynamoDB stores mappings. CDK defines all resources.

10.1 Key Features

- POST /shorten to create a short code
- GET /{code} returns HTTP 301 redirect
- GET /health for liveness checks
- Conditional writes to avoid short-code collisions
- Hit counter on successful redirects
- Least-privilege IAM and CloudWatch monitoring design
- Local zero-cost execution path for verification

10.2 Working Principle

When a client submits a long URL, the Create Lambda validates the URL, generates a base62 short code and writes an item to DynamoDB. The response includes shortCode and shortUrl. When a client opens the short URL, the Redirect Lambda reads the item and returns Location: <longUrl> with status 301. Unknown codes return 404.

10.3 Comparison with Existing System

- Existing: single server / manual setup | Proposed: serverless managed services
- Existing: fixed capacity | Proposed: automatic scaling with Lambda concurrency
- Existing: manual recovery | Proposed: managed service durability and retries
- Existing: ad-hoc setup | Proposed: CDK Infrastructure as Code
- Existing: broad permissions common in demos | Proposed: least-privilege IAM by function

11. System Architecture

Figure 1 shows the target AWS architecture. In local verification mode, DynamoDB Local and a local HTTP API replace the managed API Gateway endpoint while using the same Lambda handler code.

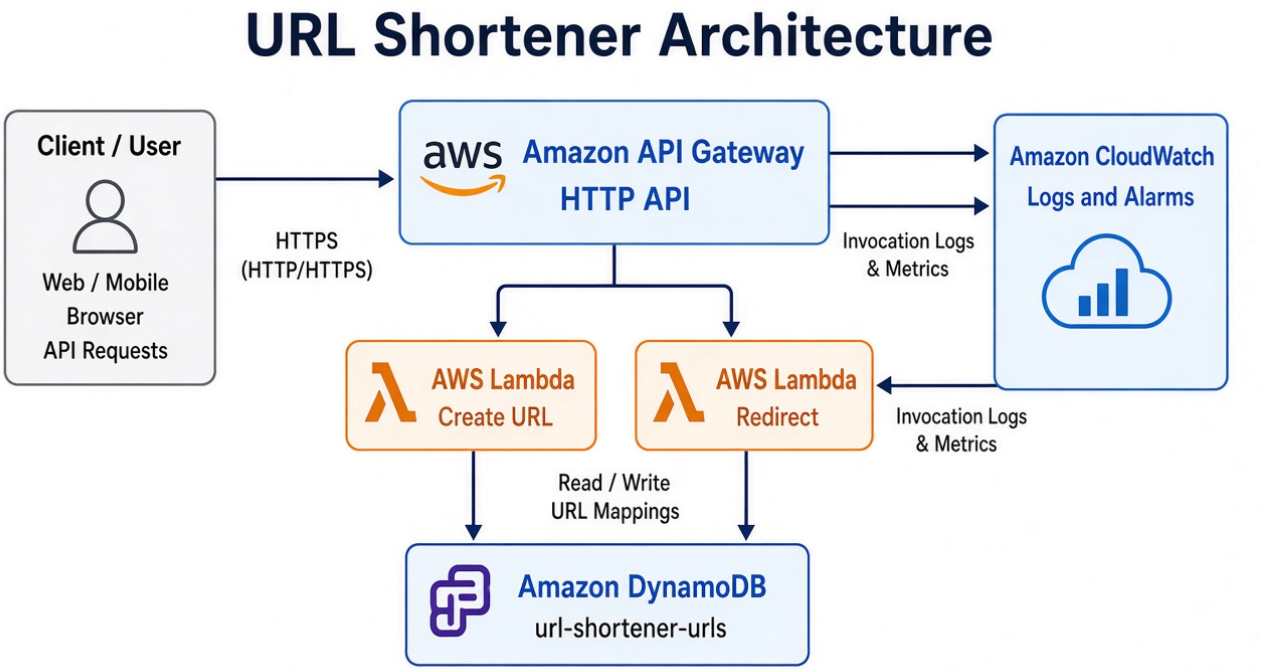


Figure 1: System architecture - Client, API Gateway, Lambda functions, DynamoDB and CloudWatch.

11.1 Components

- API Gateway HTTP API: entry point for /shorten, /health and /{code}
- Lambda CreateUrl: validation, code generation, PutItem
- Lambda Redirect: GetItem, UpdateItem hit count, 301 response
- DynamoDB table url-shortener-urls with partition key shortCode
- CloudWatch log groups and error alarms

11.2 Data Model

- shortCode (S) - partition key
- longUrl (S) - destination URL
- createdAt (S) - ISO timestamp
- hitCount (N) - redirect counter
- lastAccessedAt (S) - last redirect time

12. Technologies Used

- Amazon API Gateway (HTTP API)
- AWS Lambda (Node.js 24.x)
- Amazon DynamoDB
- AWS IAM
- Amazon CloudWatch Logs and Alarms
- AWS CDK v2 (TypeScript) / CloudFormation
- DynamoDB Local for zero-cost testing
- Jest for unit and local integration tests
- Git / GitHub for source control

13. Implementation Methodology

Step 1: Project scaffolding

Initialize an AWS CDK TypeScript application and define the UrlShortenerStack.

Step 2: DynamoDB table

Create table url-shortener-urls with partition key shortCode and on-demand billing mode in the CDK stack.

Step 3: Lambda handlers

Implement create-url and redirect handlers with URL validation, collision retry, structured logging and support for DYNAMODB_ENDPOINT for local testing.

Step 4: API Gateway routes

Wire POST /shorten, GET /health and GET /{code} using HttpApi and Lambda integrations.

Step 5: IAM least privilege

Grant PutItem only to Create Lambda; GetItem and UpdateItem only to Redirect Lambda.

Step 6: Monitoring

Attach log groups with 7-day retention and CloudWatch alarms on Lambda Errors.

Step 7: Local verification

Run DynamoDB Local, start the local API, execute unit/integration tests and smoke tests. Cloud deploy was deferred to avoid AWS charges and because root credentials must not be used for deployment.

13.1 Project Structure

- bin/url-shortener.ts - CDK application entry point
- lib/url-shortener-stack.ts - DynamoDB, Lambda, HTTP API, IAM, Logs, Alarms
- lambda/create-url/index.ts - create + health handlers
- lambda/redirect/index.ts - redirect handler
- lambda/shared/dynamo.ts - DynamoDB client factory (cloud/local)
- scripts/local-api.ts - local API Gateway stand-in
- scripts/local-setup.ps1 / local-setup-jar.ps1 - DynamoDB Local setup
- scripts/smoke-test.ps1 - end-to-end smoke tests
- test/ - Jest unit and local integration tests
- docs/ - architecture, local guide and this report

13.2 IAM Design Details

Security is a core DevOps concern. The stack avoids broad AdministratorAccess on application roles.

- Create function role: dynamodb:PutItem on the table ARN only
- Redirect function role: dynamodb:GetItem and dynamodb:UpdateItem only
- Both roles receive AWSLambdaBasicExecutionRole for CloudWatch Logs
- X-Ray put permissions are included because tracing is enabled
- API Gateway receives lambda:InvokeFunction through resource-based policy statements
- No AWS access keys are hardcoded in source files

13.3 API Examples

Create request body:

```
{ "url": "https://docs.aws.amazon.com/" }
```

Successful create response includes shortCode, shortUrl, longUrl and createdAt.

Redirect response is not JSON; it is an HTTP 301 with a Location header.

14. System Testing

Testing was performed locally against DynamoDB Local and the local API at http://127.0.0.1:3000.

14.1 Test Cases

- Health check returns HTTP 200 and {status: ok}
- Valid POST /shorten returns HTTP 201 with shortCode and shortUrl
- Invalid JSON / missing URL returns HTTP 400
- GET /{code} for existing mapping returns HTTP 301 with Location
- Unknown short code returns HTTP 404
- CDK unit tests assert table, functions, API and IAM statements

14.2 Tools

- Jest unit and local integration tests
- PowerShell smoke-test.ps1
- curl.exe for redirect header inspection
- Playwright desktop screenshots (1366x768) for report evidence

15. Results and Discussion

The following figures were captured at a standard desktop viewport (1366x768) from the working local deployment. They show end-to-end behavior of the URL shortener handlers.

15.1 Demo Console Overview

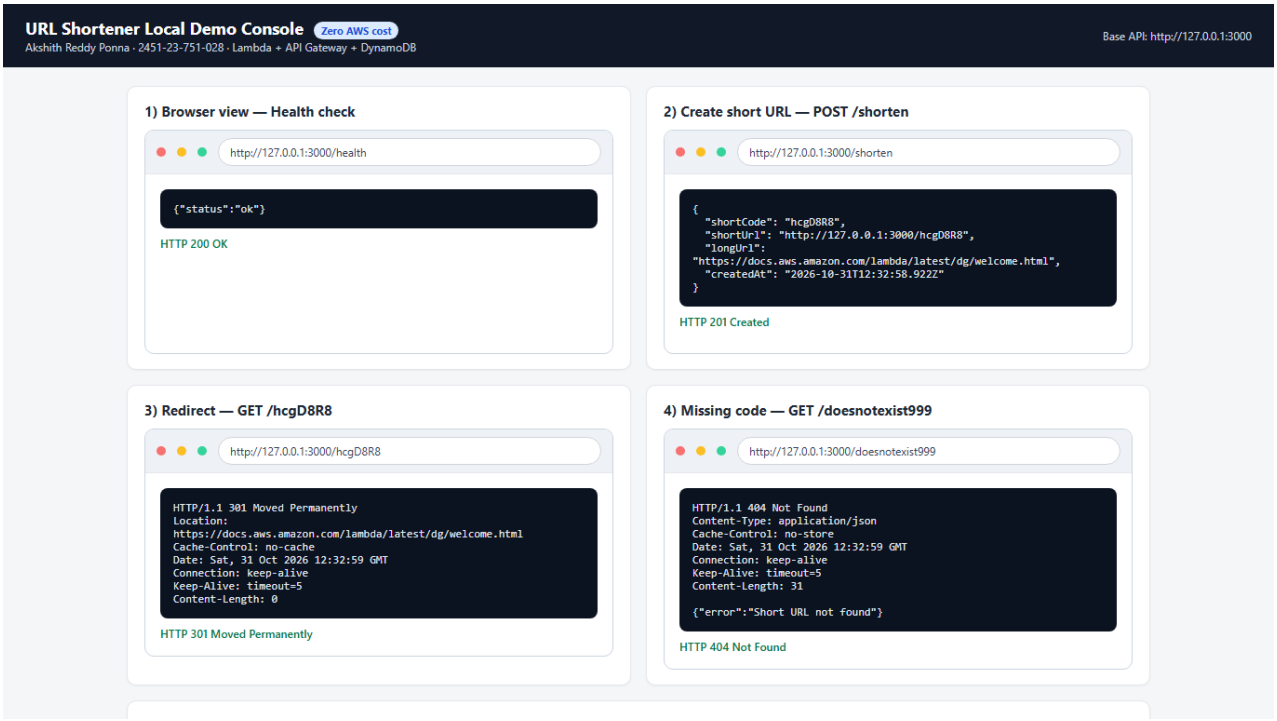


Figure 2: Local demo console showing health and create/redirect panels at normal desktop scale.

15.2 Health Endpoint

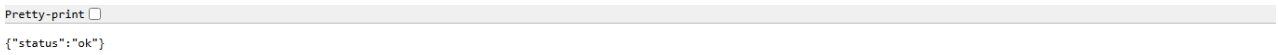


Figure 3: Browser response for GET /health returning {"status":"ok"}.

15.3 Create Short URL

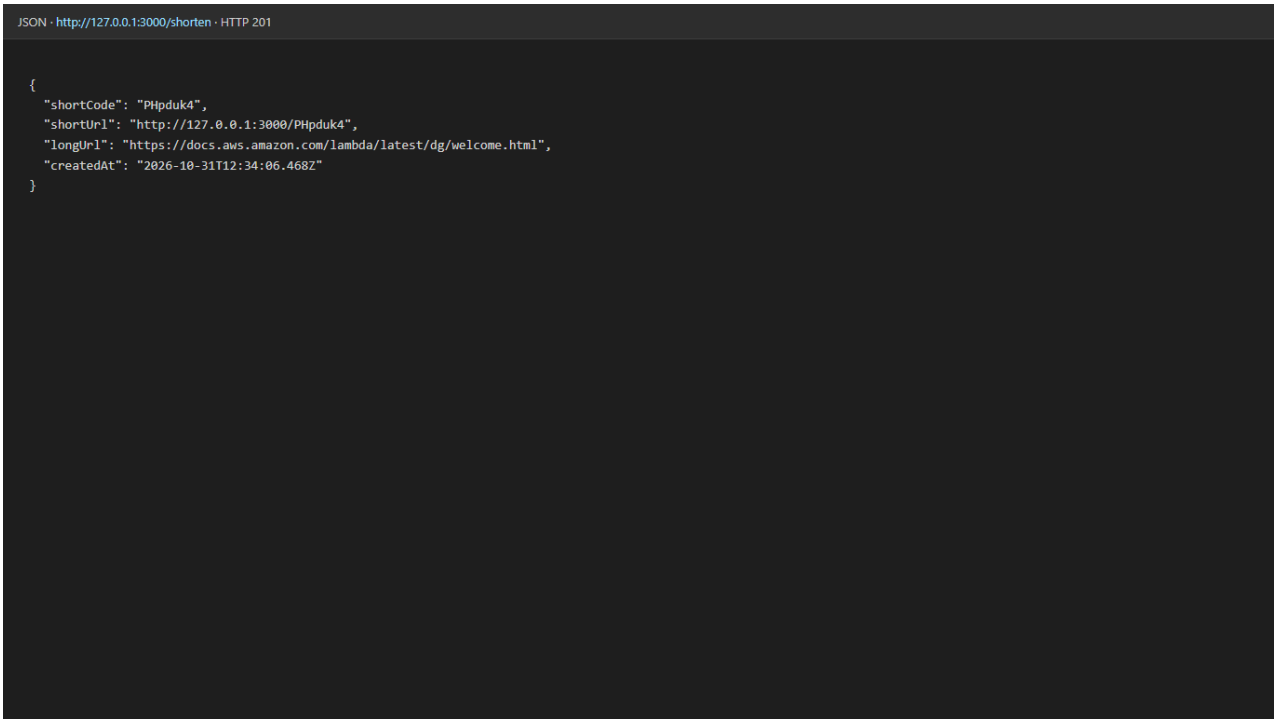


Figure 4: Successful POST /shorten response (HTTP 201) with shortCode, shortUrl, longUrl and createdAt.

15.4 Redirect Response

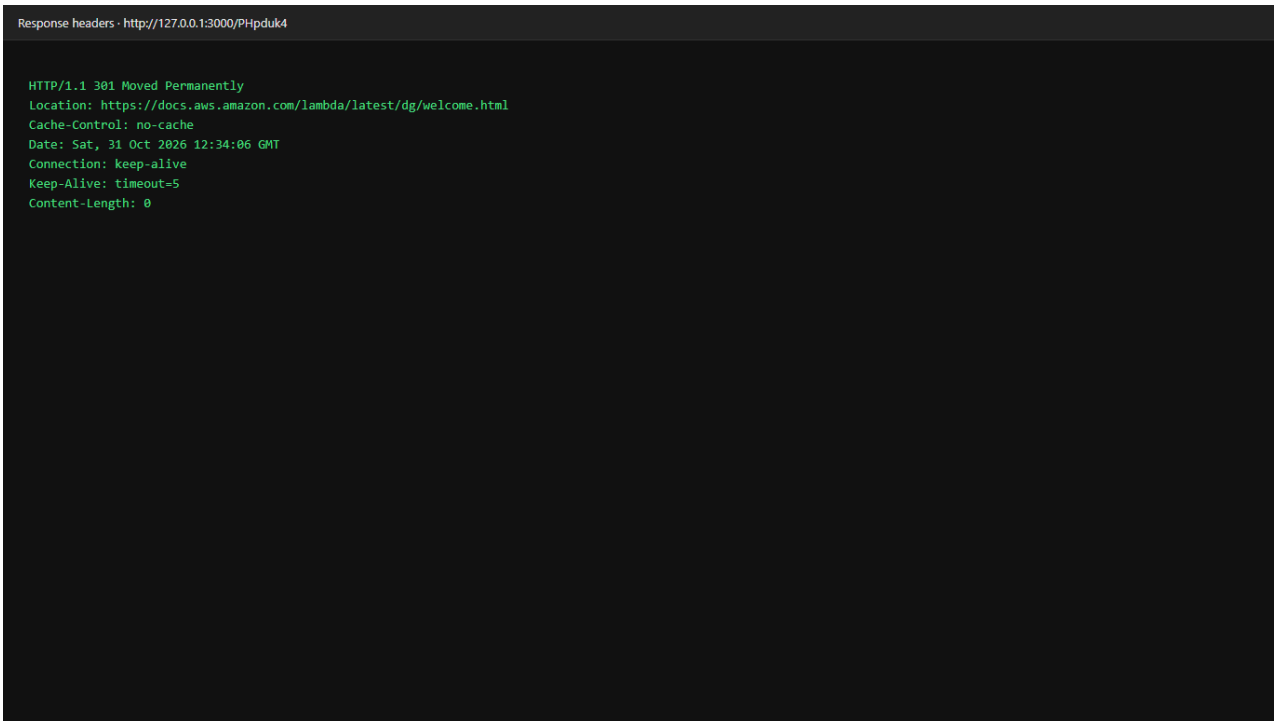


Figure 5: Redirect response headers showing HTTP 301 Moved Permanently and Location of the long URL.

15.5 Not Found Handling

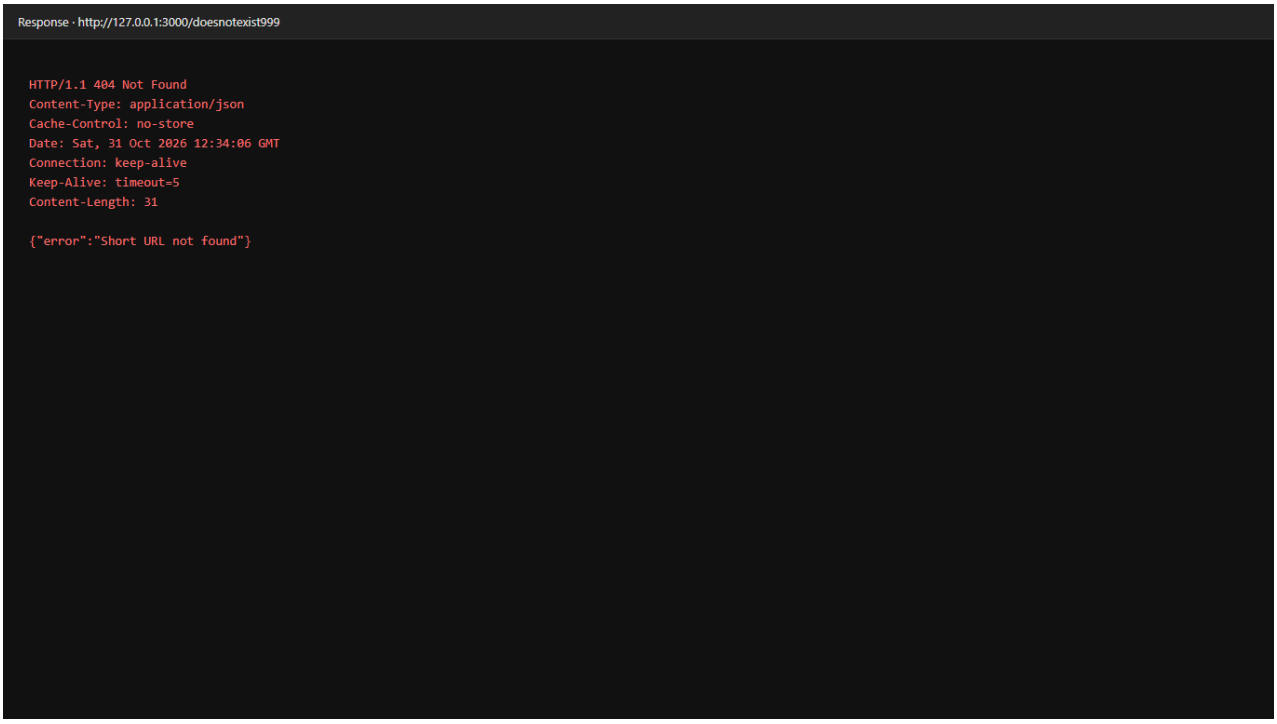


Figure 6: Unknown short code returns HTTP 404 with an error payload.

15.6 Smoke Test Terminal Output

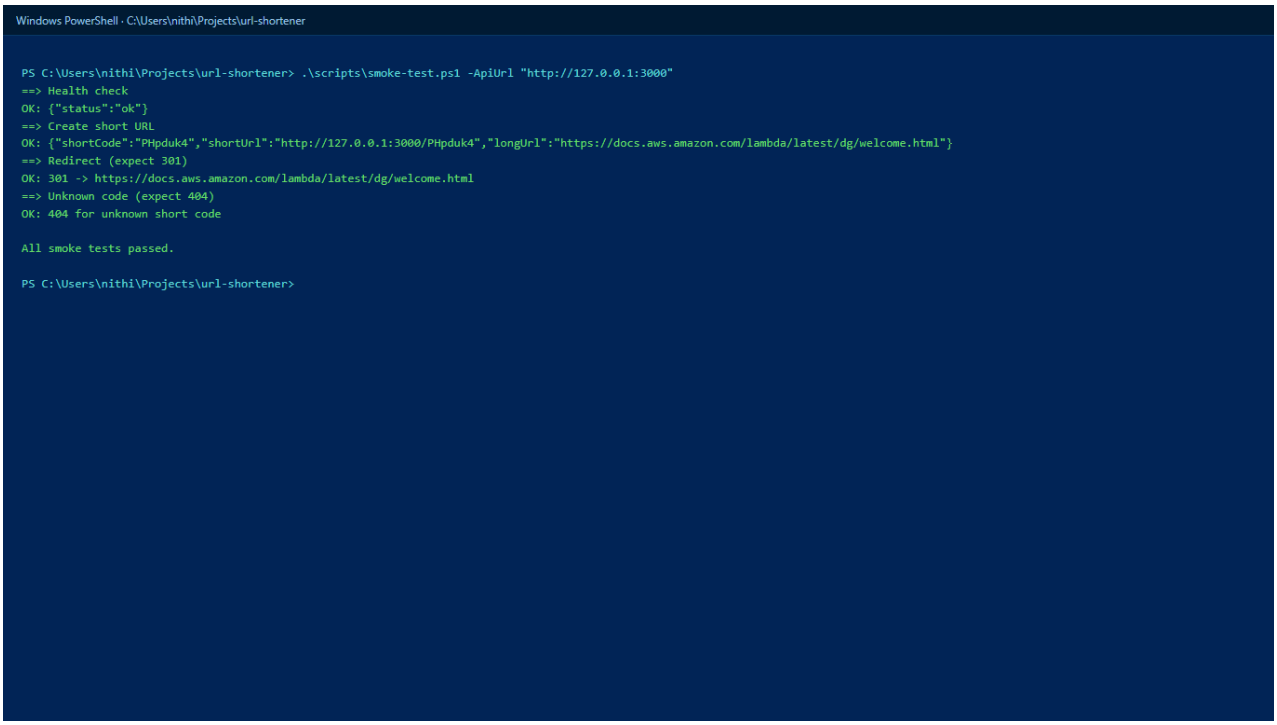


Figure 7: PowerShell smoke test output confirming health, create, redirect and 404 cases.

15.7 Source Repository

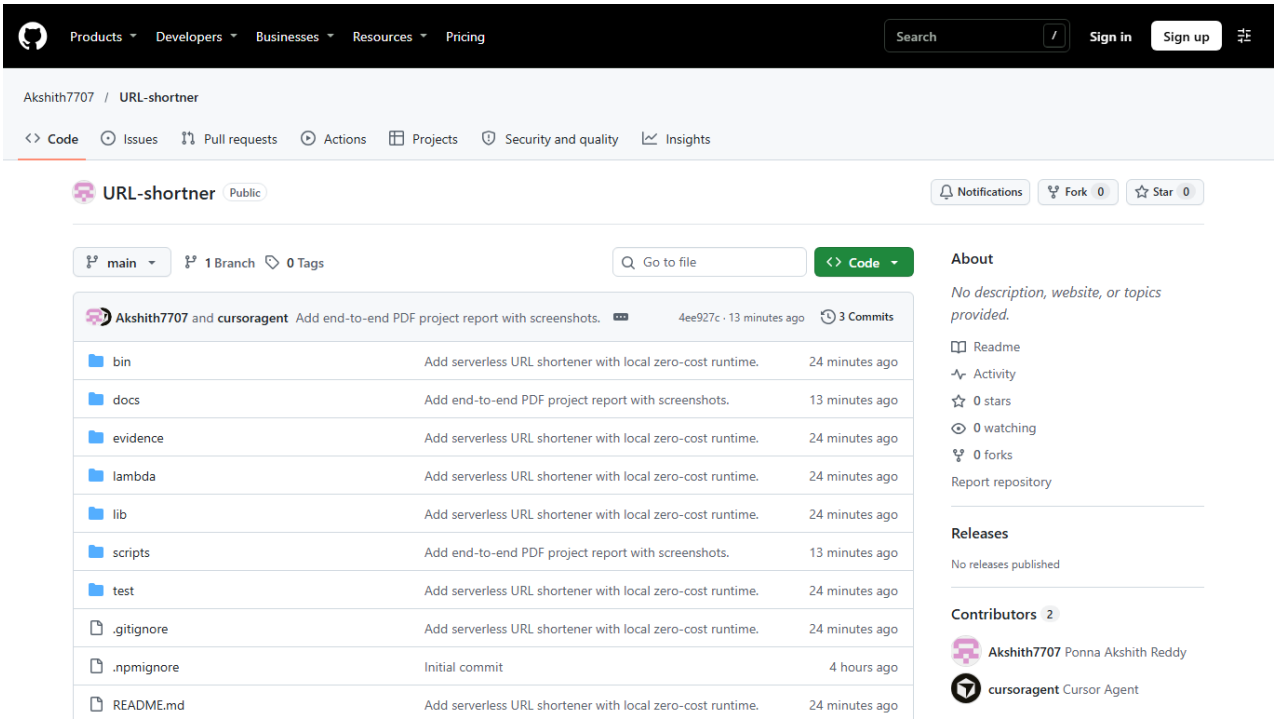


Figure 8: GitHub repository https://github.com/Akshith7707/URL-shortner containing the project source.

15.8 Summary of Observations

- Health endpoint is available and returns a clear OK payload
- Create flow stores mappings and returns a usable short URL
- Redirect flow issues HTTP 301 with the correct Location header
- Missing codes are handled with HTTP 404
- Infrastructure code synthesizes cleanly with least-privilege IAM statements
- Local verification proves functional correctness without AWS cloud charges

16. Advantages and Limitations

Advantages

- Serverless design reduces server management effort
- Pay-per-use model is cost-efficient for variable traffic
- IaC enables repeatable deployments
- Least-privilege IAM improves security posture
- Local testing path supports zero-cost evaluation

Limitations

- Cloud deploy was not executed in this submission to avoid AWS spend and root-user risk
- No custom domain or HTTPS certificate management in the current demo
- No authentication/rate limiting on the public create API
- CORS is open for demo purposes and should be restricted in production

17. Applications

- Marketing campaign links and QR codes
- Internal documentation deep-link shortening
- Social media sharing of long product URLs
- Academic and portfolio project demos
- Learning platform for serverless DevOps skills

18. Future Scope

- Deploy with a non-root IAM role after budget approval
- Add custom domain with ACM certificate
- Add API authentication (API keys / Cognito / JWT)
- Add analytics dashboard for hit counts
- Add TTL/expiry for temporary links
- Add CI/CD pipeline for CDK deployments
- Add AWS WAF for abuse protection

19. Conclusion

This project successfully designs and implements a serverless URL shortener using AWS Lambda, API Gateway and DynamoDB, with infrastructure defined in AWS CDK. The implementation includes secure IAM design, monitoring configuration, automated tests and complete documentation.

End-to-end verification was completed locally using DynamoDB Local and a local API, confirming create, redirect and error-handling behavior. The same codebase is ready for AWS deployment when a non-root identity and explicit cost approval are available.

Through this work, practical experience was gained in serverless architecture, Infrastructure as Code, least-privilege security and DevOps-style validation - core skills for Cloud Infrastructure and DevOps Engineer roles.

Declaration

I, Akshith Reddy Ponna (Roll No: 2451-23-751-028), submit this project report for the domain Cloud Infrastructure and DevOps Engineer. The project title is URL Shortener using Lambda + DynamoDB. All sections of this report are completed with project details, architecture, implementation, test evidence and references. The source code is available at https://github.com/Akshith7707/URL-shortner.

Submitted by: Akshith Reddy Ponna

Roll Number: 2451-23-751-028

Domain: Cloud Infrastructure and DevOps Engineer

Project Title: URL Shortener using Lambda + DynamoDB

20. References

1. AWS Documentation - https://docs.aws.amazon.com/

2. AWS Lambda Developer Guide - https://docs.aws.amazon.com/lambda/

3. Amazon API Gateway Developer Guide - https://docs.aws.amazon.com/apigateway/

4. Amazon DynamoDB Developer Guide - https://docs.aws.amazon.com/dynamodb/

5. AWS CDK Documentation - https://docs.aws.amazon.com/cdk/

6. Choose between HTTP APIs and REST APIs - https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html

7. DynamoDB Local - https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DynamoDBLocal.html

8. IAM best practices - https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

9. Project repository - https://github.com/Akshith7707/URL-shortner

Appendix A: How to Run Locally

- npm install
- npm run local:setup:jar
- npm run local:api
- .\scripts\smoke-test.ps1 -ApiUrl http://127.0.0.1:3000

Appendix B: Submission Links

- GitHub: https://github.com/Akshith7707/URL-shortner
- Report PDF: docs/URL_Shortener_Project_Report.pdf

Appendix C: Learning Outcomes

By completing this project, the following outcomes were achieved:

- Ability to design a serverless application architecture on AWS
- Hands-on experience with AWS CDK for Infrastructure as Code
- Understanding of least-privilege IAM for Lambda and DynamoDB
- Practical API design for create and redirect workflows
- Experience with local emulation (DynamoDB Local) for cost-safe testing
- Documentation and evidence preparation for academic submission

Appendix D: Sequence of Operations

Create flow

- Client sends POST /shorten with JSON body containing url
- API Gateway invokes Create Lambda
- Lambda validates protocol/host and rejects unsafe local targets
- Lambda generates a 7-character base62 short code
- Lambda writes item with condition attribute_not_exists(shortCode)
- On collision, Lambda retries with a new code
- Client receives HTTP 201 with shortCode and shortUrl

Redirect flow

- Client sends GET /{code}
- API Gateway invokes Redirect Lambda
- Lambda validates short-code format
- Lambda reads DynamoDB item by shortCode
- If missing, return HTTP 404
- If found, best-effort increment hitCount and return HTTP 301 Location

Local zero-cost mapping

- API Gateway -> scripts/local-api.ts on port 3000
- DynamoDB service -> DynamoDB Local on port 8000
- Same handler source files used for both local and cloud paths